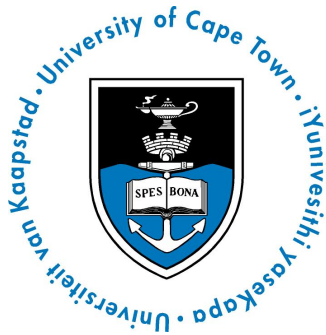


University of Cape Town



Supervised Learning Assignment 2

AirBnB Bookings Prediction

Pamela Afful

Student Number: AFFPAM001

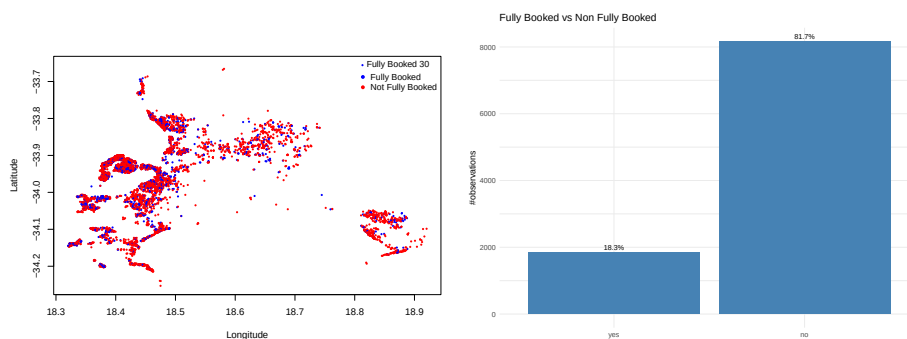
May 15, 2026

Introduction: Exploratory Data Analysis

The training data consists of 10 rows of observations across 25 features. The data is significantly imbalanced in favour of the target class at with ~81% of observations in the filly_booked_30='no' class.

Some cleaning and transforming was done on the features to make them machine learning friendly, most notably:

- Transformed all non-numerical features into factors
- Transform the **host_since_int** variable which gives the date from which the host started hostig BnBs into an interger that calculates the host's "age" in months. The feature is saved as **host_since_months_int**.



(a) Spatial Plot of BNB Listings, Fully Booked 30 Label Distribution

A simple decision tree to gauge which features could be important for determining the response variable. Since we have an imbalanced class, a weighted decision tree is used. The weights are a vector of ratios of the grouped observations counts of the different classes with the minority "yes" class. being weighted more. The ensure that the model is penalized more for misclassifying the minority class(<https://machinelearningmastery.com/cost-sensitive-decision-trees-for-imbalanced-classification/>)

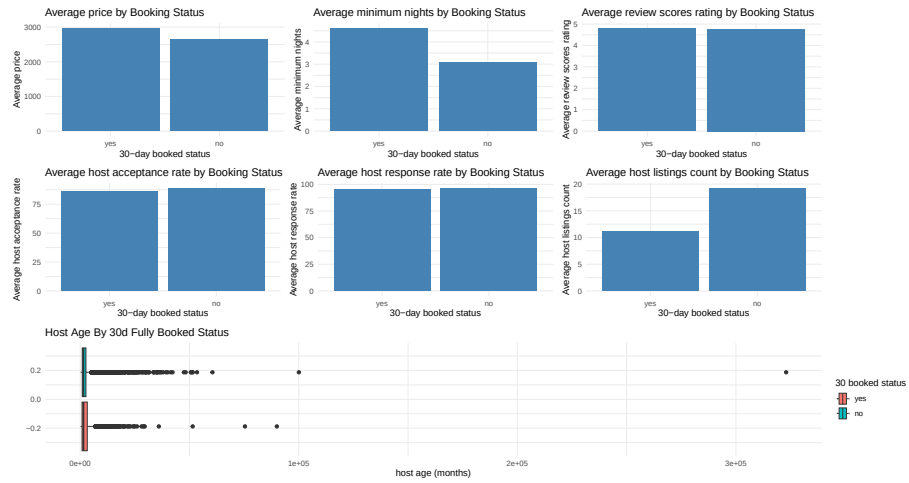


Figure 3: Averages and Distributions of Features per Target Class

When grouping by the target variable, fully_booked_30, the following key insights are observed:

- Average Price and average minimum nights are slightly higher for observations in the “yes” class, with the gap much wider for average minimum nights (~ 50 % higher than the “no” class).
- Average review scores and average response rate are fairly evenly split across the two target classes
- Average host listings are higher for the “no” class with an average of ~20 listings.
- For the host age, the median (118 months) host age is ~5% higher for the “yes” class compared to the no class. Moreover no class has a longer range of host ages, although the distributions about the median for both groups are similar; most host ages lie below the median host age.

A simple decision tree is ran to see how the nodes and targets are split in relation to the features

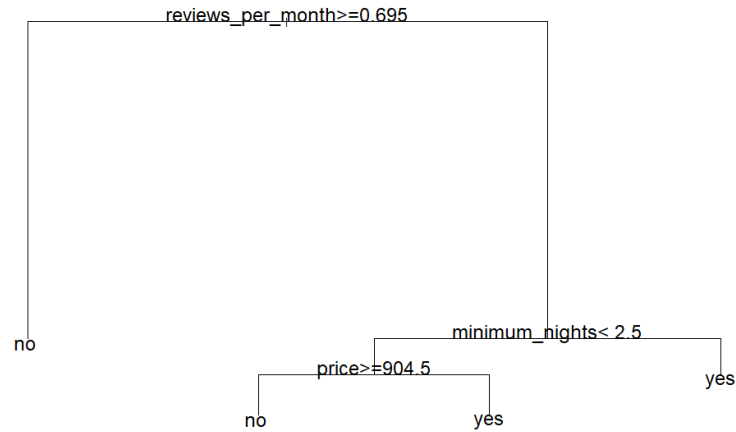


Figure 4: Initial Decision Tree Feature/ Class Splits

Features used to reduce RSS during splitting were **reviews_per_month** (the largest reduction), **minimum_nr_night**, and **price**.

BnB listings with `reviews_per_month` greater than `0.695` all had terminal nodes categorised as “no” for the response variable. Price and minimum nights were determining factors for listings with a minimum number of nights less than `2.5`, which ended in “yes”. Further splitting this region, for listings where the minimum was greater than `2.5`, listings with prices greater than `905.5` were also categorised as “yes”.

Model Training

Baseline Model Vanilla Logistic Regression

Table 1: Vanilla Logistic Regression Summary

Characteristic	log(OR) [†]	SE
(Intercept)	-26*	10.2
host_response_time	—	—
within an hour	—	—
within a few hours	-0.18*	0.073
within a day	-0.42***	0.097
a few days or more	-0.47	0.324
host_response_rate	0.00	0.003
host_acceptance_rate	0.00*	0.002
host_is_superhost	—	—
t	—	—
f	-0.07	0.060
host_listings_count	0.01***	0.001
host_verifications	—	—
phone_only	—	—
phone_email	-0.05	0.097
email_only	-2.0	1.50
latitude	-0.56*	0.278
longitude	0.54*	0.252
room_type	—	—
Entire home/apt	—	—
Private room	0.15	0.103
accommodates	0.01	0.030
bathrooms	0.08	0.047
bedrooms	-0.12*	0.056
beds	0.00	0.027
price	0.00	0.000
minimum_nights	0.01	0.011
maximum_nights	0.00	0.000
minimum_nights_avg_ntm	-0.02	0.011
maximum_nights_avg_ntm	0.00	0.000
number_of_reviews	0.00	0.001
reviews_per_month	0.67***	0.054
review_scores_rating	-0.23**	0.085
instant_bookable	—	—
t	—	—
f	-0.21**	0.073
host_since_months_int	0.00***	0.001

[†]p<0.05; **p<0.01; ***p<0.001₄

Abbreviations: CI = Confidence Interval, OR = Odds Ratio, SE = Standard Error

No. Obs. = 10,000

- Significant Features:
 - The number of reviews per month has the highest and significant effect on the probability of the target variable, a unit increase in the number of reviews per month increases the likelihood of an Air BnB not being fully booked by ~95% (Calculation $e^{**\beta}$, where $\beta=0.67$).
 - Despite the number review being positive, the actual ratings does not show a significant effect the the log likelihood of the target class.
 - Latitude and Longitude- location places a significant role BnB demand and the likelihood of a BnB being fully booked- although the signs for both features are different
 - Although features `host_acceptance_rate` is significant, `host_since_months_int` and `host_acceptance_rate` are significant they does not seem to have an extremely negligible effect on the likelihood of the positive class(

- target='no').
- Host Listings Count
- Review Score Ratings
- Instant bookable=F,
- Host Since (Measured in months)
- Notable Insignificant Features:
 - Most interestingly the price did not show a stritically significant effect on the (log) lilkiehood of the target class (although this was a key feature used the decision tree above).

L1 Regularization

L1 Regularization with using K-folds validation is now trained on the data with AUC used as the cross validation metric.

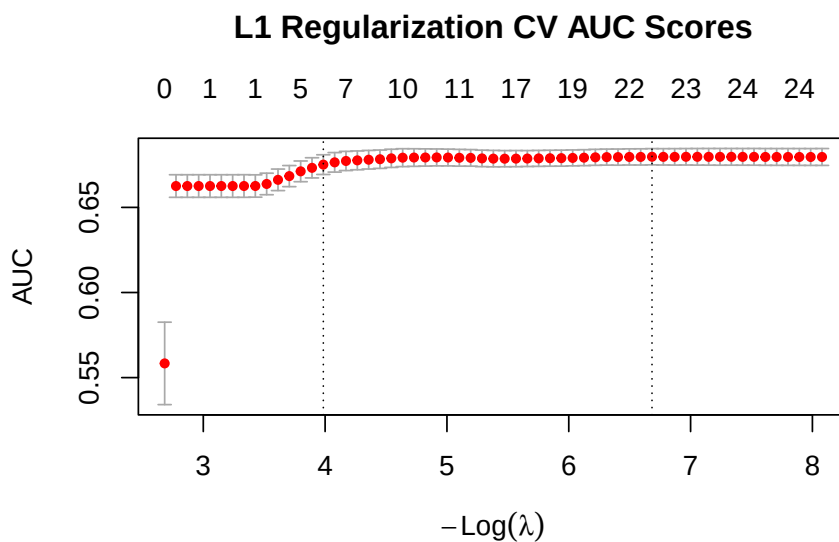


Figure 5: CV AUC Scores Per Log Lambda Values

The best CV AUC score is obtained at at 0.018 with associated 6 non-zero coefficients (out of the 25) retained by the model.

Most notably:

- The number reviews per month maintains its position as having the biggest marginal impact on the (log) likelihood of the target.

- Location variables, location and longitude are penalized to zero.
- For the categorical feature, host_response, the host_response_time_within_a_day was the only retained category in the regression
- Minimum_nights_avg_ntm was not penalised in the L1 regression even though it was not significant in the vanilla model

Table 2: L1 Logistic Regression Results, penalized variables in red

Feature	Coefficient
(Intercept)	1.1863048
host_response_timewithin a few hours	0.0000000
host_response_timewithin a day	-0.1219905
host_response_timea few days or more	0.0000000
host_response_rate	0.0000000
host_acceptance_rate	0.0000000
host_is_superhostf	0.0000000
host_listings_count	0.0022551
host_verificationsphone_email	0.0000000
host_verificationsemail_only	0.0000000
latitude	0.0000000
longitude	0.0000000
room_typePrivate room	0.0000000
accommodates	0.0000000
bathrooms	0.0000000
bedrooms	0.0000000
beds	0.0000000
price	0.0000000
minimum_nights	0.0000000
maximum_nights	0.0000000
minimum_nights_avg_ntm	-0.0022060
maximum_nights_avg_ntm	0.0000000
number_of_reviews	0.0000000
reviews_per_month	0.4268373
review_scores_rating	0.0000000
instant_bookablef	-0.0642674
host_since_months_int	-0.0004212

Random Forest

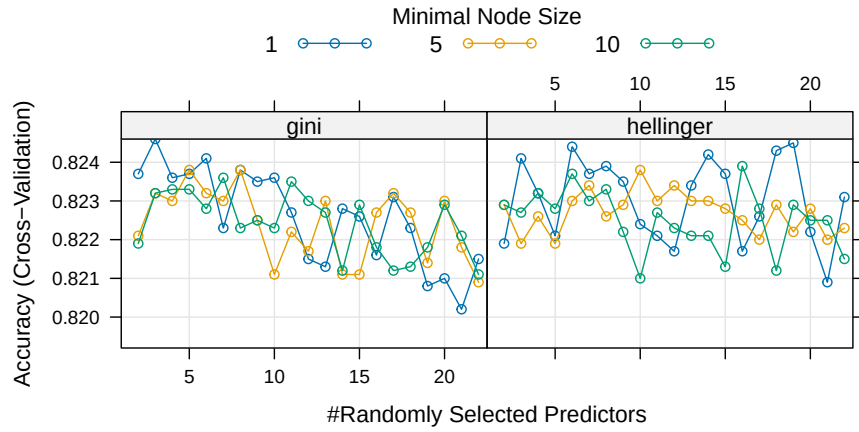


Figure 6: CV Accuracy Scores vs Hyparameters

Table 1: Best RF HyperParameters

mtry	splitrule	min.node.size
7	3 gini	1

The best CV accuracy score is achieved at the parameters shown below

Boosting Model

The gbm model is trained on the non-zero features from the L1 regularized model to speed up training,, as it has been reasonably established that the majority of these are not meaningful to the model.

Also, a custom log-loss function is used as the primary metric for the the cross validation training- thus the hyper parameters that yield the lowest log loss function are chosen as the best during training.

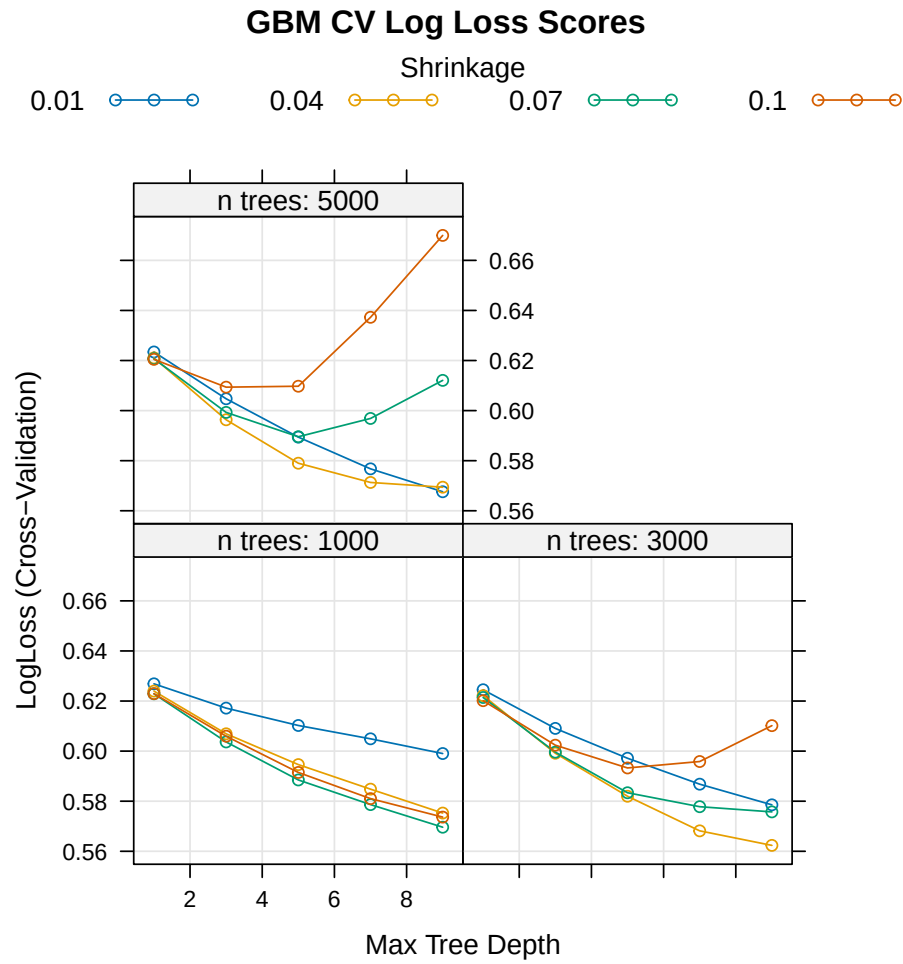


Figure 7: GBM CV Log Loss Values vs Hyperparameters

The best hyper parameters is acquired at the hyperparameter combinations below:

Table 2: GBM Best HyperParameter Combination

	nr trees	interaction depth	shrinkage
60	5000	9	0.1

```

# using all features comparability

library(doParallel)
library(foreach)
set.seed(123)

# ctrl = trainControl(method = 'cv', number = 10, verboseIter = F,
ctrl = trainControl(method = 'cv', number = 10, verboseIter = F,

gbm_grid <- expand.grid(n.trees = c(500,1000,1500),
                        interaction.depth = seq(1, 5, 1),
                        shrinkage = c(0.005, 0.01),
                        n.minobsinnode = 10)

#CV for hyper parameter tuning
# n_cores <- parallel::detectCores() - 1 # leave one core free
# cl <- makeCluster(n_cores)
# clusterEvalQ(cl, library(Metrics))
# registerDoParallel(cl)
# clusterExport(cl, varlist = c("log_loss"))
#
# gbm_all_gridsearch <- train(fully_booked_30 ~ ., data = listings_data_cleaned_features,
#                             method = 'gbm',
#                             distribution = "bernoulli", #
#                             trControl = ctrl,
#                             verbose = T,
#                             tuneGrid = gbm_grid,
#                             metric = "LogLoss",
#                             weights=weights_vector)
#
# stopCluster(cl)
# registerDoSEQ()

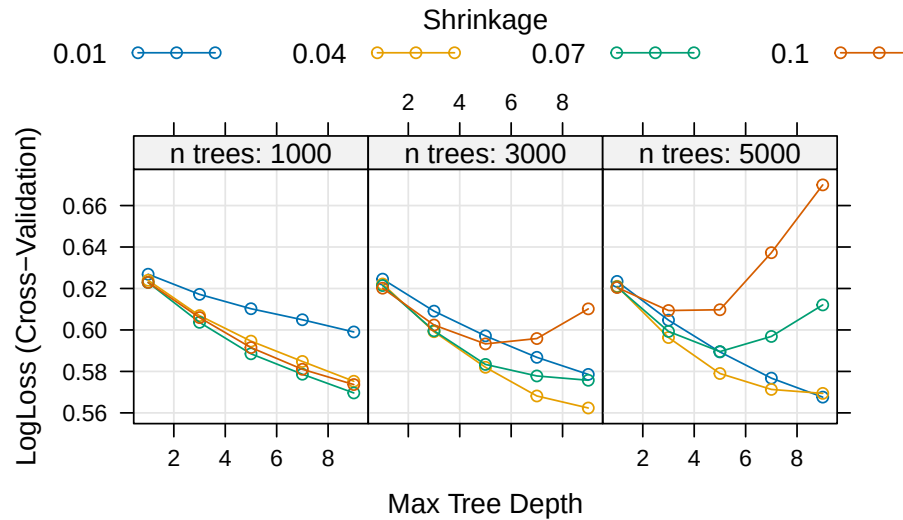
#save(gbm_all_gridsearch , file = 'gbm_all_gridsearch.Rdata')

saveRDS(rf_gridsearch, file = "caret_rf_grid_results_cv_weights.rds")

#saveRDS(rf_gridsearch, file = "caret_rf_grid_results_cv_weights.rds")

#load('gbm_all_gridsearch.Rdata')

```



```
kable(gbm_gridsearch$bestTune %>% dplyr:: select(n.trees,interaction.depth,shrinkage) %>%
  rename(`nr trees`=n.trees,`interaction depth`=interaction.depth)
  , caption = 'GBM Best HyperParameter Combination')
```

Table 3: GBM Best HyperParameter Combination

	nr trees	interaction depth	shrinkage
60	5000	9	0.1

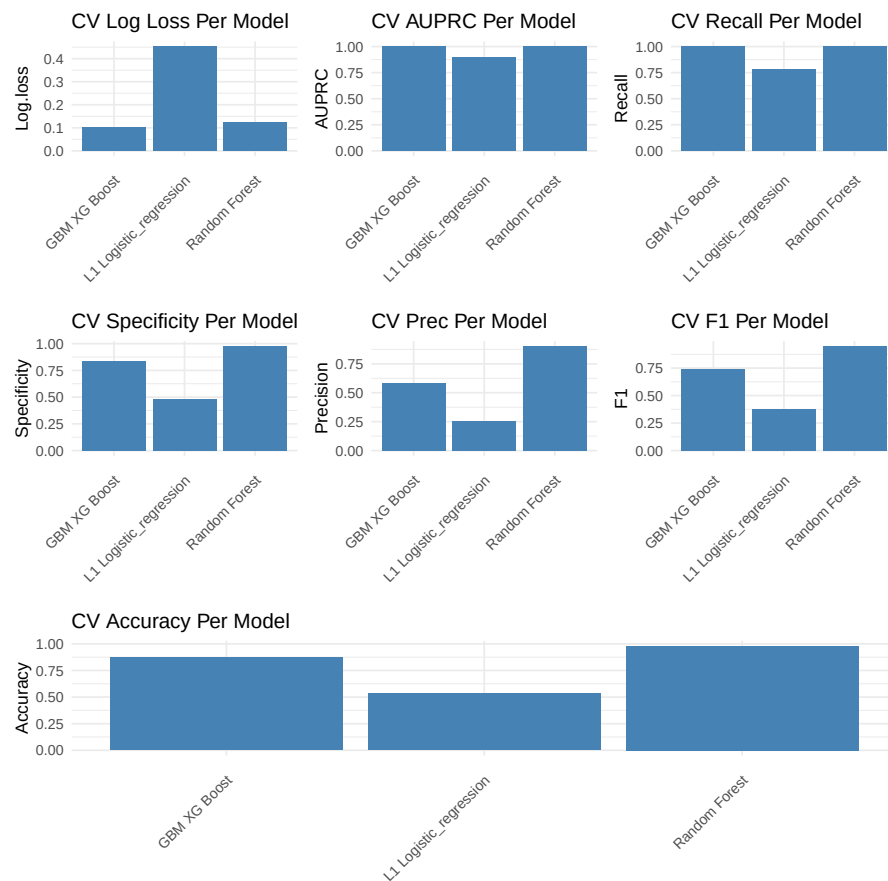
Question 3- Model Evaluation and Selection

a)

[1] 0.8166

	model	Log.loss	AUPRC	Recall	Specificity	Precision
1	L1 Logistic_regression	0.4525573	0.8996259	0.7770254	0.4796749	0.2512258
	F1 Accuracy					
1	0.3796695	0.5342054				

	model	Log.loss	AUPRC	Recall	Specificity	Precision	F1
1	GBM XG Boost	0.1035011	0.9998888	1	0.8408813	0.5846645	0.7379032
	Accuracy						
1	0.87						



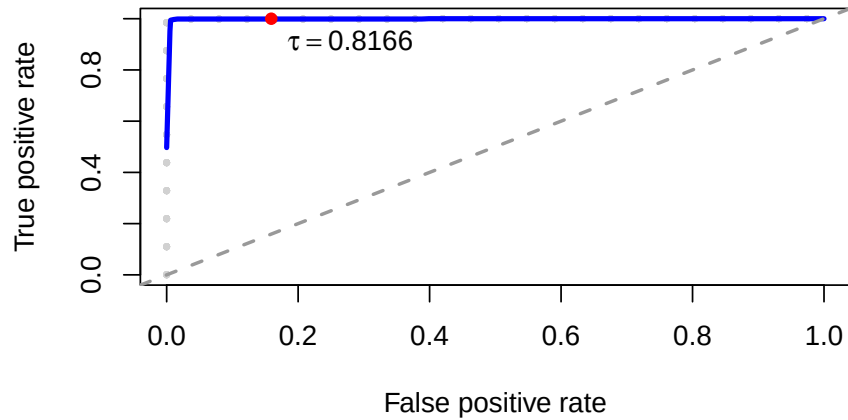
Boosting tree is the best model, given all the metrics:

- Lowest penalty of negative predictions from log loss- primary metric
- Best AUPRC- best metric for imbalanced data set

[1] 1

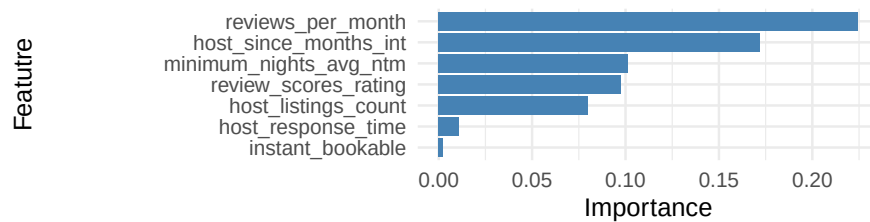
[1] 0.1591187

Boosted Tree 10 Fold CV ROC

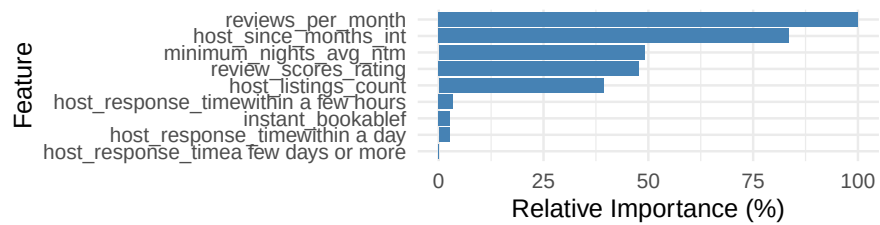


Section 4- Interpretation

Permutation Variable Importance



GBM Impurity Variable Importance



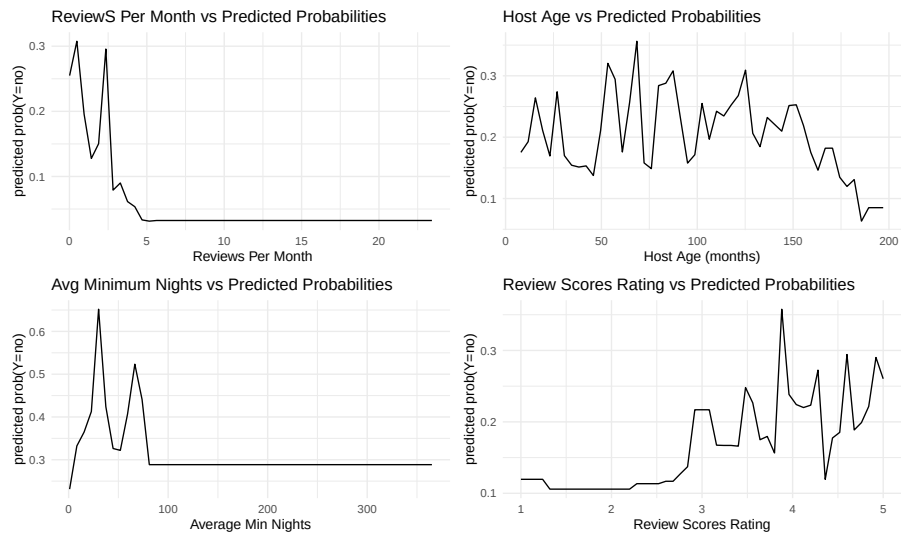


Figure 8: Partial Dependence Plots for Top 4 Features